



MINISTERIO DEL PODER POPULAR PARA LA EDUCACIÓN SUPERIOR

UNIVERSIDAD SIMÓN BOLÍVAR

COORDINACIÓN DE CICLO BÁSICO

SARTENEJAS, DISTRITO CAPITAL.

Informe

Etapas 3 – Analizador de Contexto

Estudiantes:

Alejandro Villamizar 22-10439

Francisbell Graterol 22-10177

Carlos Bisogno 22-10041

Sartenejas, septiembre 2025.

1. Introducción

Este informe describe la formulación e implementación del archivo `syntax.py` encargado del análisis de contexto del lenguaje Bot, en base al análisis sintáctico creado por `parser.py`, en donde se ha construido el Árbol Sintáctico Abstracto (AST) del programa fuente. El objetivo de esta etapa es verificar que el programa sea semánticamente correcto.

2. Estructura de Datos

El analizador utiliza tres clases que, en conjunto, implementan la tabla de símbolos y permite tener alcances anidados. Las clases son:

Symbol: representa una entrada individual de la tabla de símbolos: nombre, tipo y clase (variable de bot, parámetro, etc.).

Scope: modela un alcance mediante un diccionario de símbolos y una referencia a su ámbito padre. Al tener un encadenamiento de alcances permite hacer una búsqueda de identificadores desde el nivel local hacia “arriba” en los alcances externos.

SymbolTable: administra la pila de alcances (scopes) y acumula los errores semánticos.

El uso de una pila de alcances (*`self.scopes`* en `SymbolTable`) es una decisión importante para esta sección, cada vez que el analizador entra en un bloque con alcance propio (el cuerpo de un `if`, de un `else`, de un `while` o de un comportamiento `on`), se invoca *`tabla.push()`* para crear un alcance hijo, y al salir se invoca *`tabla.pop()`* para descartarlo. Gracias a esto, una variable declarada dentro de un bloque no es visible fuera de él, y una variable con el mismo nombre puede redeclararse en un bloque interno sin generar conflicto con el alcance externo.

3. Análisis de Expresiones

Esta función implementa la verificación de tipos mediante recorrido recursivo del AST de una expresión. Por cada tipo de nodo de expresión se aplica una regla distinta:

- **NumberNode y BoolNode:** son los casos base de la recursión; su tipo es directamente `int` o `bool`.

- **VariableNode:** si el nombre es la palabra reservada 'me', se verifica la tabla actual *in_behavior*; si es una variable común, se busca en la tabla de símbolos con *tabla.lookup()*, que recorre la cadena de alcances padre. Si no se encuentra, se reporta el error de variable no declarada y se le asigna el tipo 'unknown' para no propagar errores en cascada.
- **AritOpNode, RelOpNode y BoolOpNode:** analizan recursivamente ambos operandos y verifican que sus tipos coincidan con lo exigido por el operador (enteros para operaciones aritméticas y relacionales, booleanos para operaciones lógicas).
- **UnaOpNode:** distingue entre negación lógica (~, exige bool) y negación aritmética (-, exige int).

Un aspecto clave es el uso del tipo 'unknown' como valor comodín. Cuando una variable no declarada se utiliza dentro de una expresión más grande, propagar un error de tipo sólo añadiría ruido, ya que el error real ya fue reportado en el punto de la declaración faltante. Por ello, las comprobaciones de tipo aceptan 'unknown' como compatible con cualquier tipo esperado, evitando una cascada de errores redundantes derivados de una única causa.

5. Análisis de Sentencias: *visitar_sentencia*

Esta función recorre el AST a nivel de sentencias y aplica, según el tipo de nodo, la regla semántica correspondiente:

- **IfNode / WhileNode:** exigen que la condición sea de tipo bool y abren un nuevo alcance para el cuerpo.
- **StoreNode y DropNode:** delegan en *analizar_expresion* para verificar la expresión asociada a la instrucción.
- **ContNodes:** verifica que cada variable listada este previamente declarada.
- **ReadNode y CollectNode:** verifican que la variable destino de la lectura o recolección exista en la tabla de símbolos.
- **MovimientoNode:** exige que la cantidad de movimiento sea de tipo entero.

- OnNode: representa un bloque de comportamiento del bot. Antes de recorrer su cuerpo, activa la bandera `in_behavior` (guardando el valor previo) para permitir el uso de 'me' dentro del bloque, y la restaura al salir; además abre y cierra un alcance propio para las variables locales del comportamiento.
- EstListNode: actúa como nodo contenedor, delegando el análisis a cada sentencia de la lista.

6. Análisis Global: *analizar_contexto*

Esta función maneja el análisis completo del programa y define el orden en que se procesan las declaraciones:

- Primero, para cada bot declarado (`BotDefNode`), registra en la tabla de símbolos todas las variables asociadas a ese bot con la clase '`bot_variable`'.
- Luego, si el bot define comportamientos (bloques `on`), los analiza mediante `visitar_sentencia`.
- Finalmente, recorre el controlador principal del programa (*programa.controlador*), que representa la lógica central fuera de los bots.

El orden de declarar primero todas las variables de un bot y después analizar sus comportamientos garantiza que, dentro de un bloque `on`, se pueda hacer referencia a cualquier variable del bot sin importar en que orden fueron escritas en el código fuente, evitando falsos errores de 'variable no declarada' por dependencias de orden textual.

6. Manejo de Errores

Todos los errores semánticos se acumulan como cadenas de texto en `tabla.errors`, con mensajes que identifican el tipo de problema (redeclaración, variable no declarada, error de tipo, uso invalido de 'me', etc.). Si la lista de errores no está vacía, se imprimen todos antes de continuar, y solo si el análisis de contexto no encontró errores se procede a imprimir el árbol sintáctico.